Two-Level Graph Representation Learning with Community-as-a-Node Graphs

Jeong-Ha Park[1], Kisung Lee[2], and Hyuk-Yoon Kwon[3,*]
[1,3]Graduate School of Data Science, Seoul National University of Science and Technology, Seoul, South Korea
[2]Division of Computer Science and Engineering, Louisiana State University, Baton Rouge, USA
[*] Corresponding: hyukyoon.kwon@seoultech.ac.kr

2023 IEEE International Conference on Data Mining (ICDM) | 979-8-3503-0788-7/23/$31.00 ©2023 IEEE | DOI: 10.1109/ICDM58522.2023.00158

***Abstract*—In this paper, we propose a novel graph representation learning (GRL) model that aims to improve both representation accuracy and learning efficiency. We design a Two-Level GRL architecture based on the graph partitioning: 1) local GRL on nodes within each partitioned subgraph and 2) global GRL on subgraphs. By partitioning the graph through community detection, we enable elaborate node learning in the same community. Based on Two-Level GRL, we introduce an abstracted graph, *Community-as-a-Node Graph* (*CaaN*), to effectively maintain the high-level structure with a significantly reduced graph. By applying the CaaN graph to local and global GRL, we propose *Two-Level GRL with Community-as-a-Node* (*CaaN 2L*) that effectively maintains the global structure of the entire graph while accurately representing the nodes in each community. A salient point of the proposed model is that it can be applied to any existing GRL model by adopting it as the base model for local and global GRL. Through extensive experiments employing seven popular GRL models, we show that our model outperforms them in both accuracy and efficiency.**

***Index Terms*—graph representation learning, community detection, representation accuracy, learning efficiency**

I. INTRODUCTION

A. Backgrounds

Graph representation learning (GRL) has become an essential solution in the field of graph-based mining and machine learning to effectively perform various graph analysis tasks, such as node classification, link prediction, and node clustering [1]. GRL transforms graph data into low-dimensional vector spaces, making it easier to analyze and process complex graphs. However, the challenge lies in preserving relationships between nodes (i.e., local features) and their topological structures (i.e., global features) while learning the graph data.

In this study, we deal with node embedding-based GRL and demonstrate its applications to both node- and edge-based analysis. The deep learning (DL)-based node embeddings have shown promising results across various input graphs. They can be divided into 1) random walk-based and 2) graph neural network (GNN)-based models [2]. Random walk-based models [3] generate node sequences by randomly visiting nodes in the graph and applying probability models. However, random walks cannot effectively represent the global patterns of the entire graph [4]. Moreover, they cannot consider node and edge properties such as attributes or weights. Therefore, GNN-based models have emerged, which can capture both global structural patterns and node properties [5]. Although they improve the representation accuracy of GRL, they require significant computation costs due to the inherent complexity of neural network models.

Efficient processing is another essential criterion for representing complex real-world graph datasets. Parallel processing accelerates it by partitioning the entire graph into subgraphs and processing them in parallel. Previous studies have focused on load balancing and communication costs, but they did not consider the accuracy of graph representations [6]. Therefore, to design a comprehensive graph representation method, it is crucial to consider both the processing efficiency and the representation quality.

B. Main Idea

In this paper, we propose a novel GRL model, *Two-Level GRL with Community-as-a-Node* (simply, *CaaN 2L*), which enhances both the representation accuracy and the learning efficiency of GRL. To achieve this, we first design a Two-Level GRL architecture through graph partitioning: 1) local GRL on nodes within each partitioned subgraph and 2) global GRL on subgraphs. To maintain the semantics of each subgraph, we use the communities identified by community detection as the subgraphs. Because communities are groups of nodes with similar characteristics, nodes within the community tend to be more closely related to each other rather than to nodes outside the community [7]. Therefore, to improve the representation accuracy, we treat each community as a semantic partitioning unit. We learn the elaborate relationship between nodes within the same community in each local GRL and the topological structures between communities in the global GRL. Moreover, by using the communities as physical partitioning units and performing parallel processing, we improve learning efficiency.

In Fig. 1, we first introduce two primitive methods based on Two-Level GRL. Fig. 1 (a) shows a primitive method, Accurate 2L, which aims to maximize accuracy. With the original GRL, it additionally runs local GRL for each community to elaborately learn the nodes belonging to the same community. Accurate 2L only targets major communities above a certain size because tiny communities are less effective in representation learning. With the community-based local GRL, Accurate 2L can improve the accuracy of the original GRL. However, its critical limitation is that it increases the learning time because it requires local GRL in addition to the original GRL. Fig. 1 (b) shows another primitive method, Efficient 2L,

979-8-3503-0788-7/23/$31.00 ©2023 IEEE
DOI 10.1109/ICDM58522.2023.00158

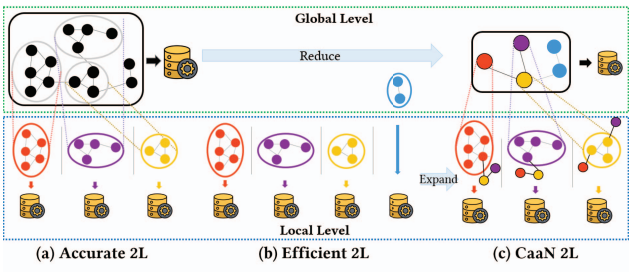


Fig. 1: The basic concept of CaaN 2L.

which aims to maximize efficiency. It assigns all nodes into communities and processes them in parallel, accelerating the learning time. However, it fails to effectively learn the global structure, thereby having limited representation accuracy.

Fig. 1 (c) illustrates the main idea of our proposed model, CaaN 2L, which aims to achieve both the accuracy of Accurate 2L and the efficiency of Efficient 2L. The most important concept is to capture the global graph patterns effectively without utilizing the entire original graph. To achieve this, we introduce an abstracted graph, *Community-as-a-Node Graph* (simply, *CaaN*), which represents each community as a representative node. The CaaN graph captures not only the relationship between major communities but also the connections between these communities and nodes outside major communities. By using this graph, we can effectively represent the relationships in the graph while significantly reducing the learning overhead compared to using the original graph.

C. Highlights

We summarize our salient points as follows:

1) Our model can be applied to any existing GRL models by using them as the base model for both local and global GRL. While maintaining the advantages of the existing models, we further improve their representation accuracy.
2) Our model successfully addresses the limitations of the existing DL-based GRL models. To capture global graph structures that cannot be considered in random walk-based models, we introduce a CaaN graph. In addition, we reduce the model complexity associated with GNN-based models by transforming the GRL problem on a large-scale graph into multiple small-sized GRL problems that can be processed in parallel.
3) Through extensive experiments using four real datasets, we show that our model outperforms existing GRL models in terms of accuracy and efficiency. In link prediction and node classification tasks, our model demonstrates significant improvements in accuracy and learning efficiency compared to existing GRL models.

II. Problem Definition

We are given an input graph $G = (V, E)$, where V is a set of nodes, and E is a set of edges with $(v_i, v_j) \in E$, indicating an edge between v_i ($\in V$) and v_j ($\in V$). We consider both cases including or not including node attributes, covering both random walk-based and GNN-based models. Specifically, G has an attribute matrix A with the shape of $(|V| * L)$, where $|V|$ and L indicate the numbers of nodes and node attributes, respectively. GRL represents each node v_i in G as a low dimensional vector denoted as $\mathbf{v}_i \in \mathbb{R}^d$, in which d represents the dimension of a vector space $\mathbb{R}$, by learning the features in G. In the representation space, GRL learns a mapping function $f(v_i) = \mathbf{v}_i$, which converts each node v_i into d-dimensional vector $\mathbf{v}_i$. As output, we obtain an embedding matrix with the shape of $(|V| * d)$ consisting of the results of learning each v_i in V.

We partition the entire graph G into k subgraphs $C = \{c_1, ..., c_k\}$, which are disjointed from each other. We define communities in G as the subgraphs. Because the community is determined by connection strength between nodes, the number of nodes for each subgraph may quite vary. We may have many small subgraphs, each with a single or very few nodes, which are not effective for GRL due to little structural information. As a result, we distinguish communities that satisfy a certain size from those that do not. Specifically, according to a threshold value for the subgraph size, ρ, we define a set of major communities $C_{major} = \{c_1, ..., c_m\}$, where each community c_i in C_{major} satisfies $|c_i| \geq \rho$, in which $|c_i|$ indicates the number of nodes in c_i, and a set of minor communities $C_{minor} = \{c_{m+1}, ..., c_k\}$ not belonging to C_{major}. We denote the numbers of nodes belonging to C_{major} and C_{minor} as $|V_{major}|$ and $|V_{minor}|$, respectively.

III. Proposed Model

A. Core Concept

In this paper, we propose a new GRL model, called Two-Level GRL with *Community-as-a-Node* (simply, *CaaN 2L*). CaaN 2L aims to maintain the advantages of efficiency in Efficient 2L while preserving the accuracy of Accurate 2L. To achieve this, we introduce a *Community-as-a-Node Graph* (simply, *CaaN)* (See Definition 1) that abstracts each community as a representative node. The CaaN graph effectively captures the topological relationships of the graph in terms of the communities. By employing the CaaN graph at both the local and global levels, we can effectively represent the relationships between communities and nodes while significantly reducing the learning overhead compared to the original graph.

Definition 1 (*Community-as-a-Node Graph (CaaN)*)**.** *The CaaN graph represents a community C_i as a super-node only if C_i is a major community (i.e., its size is greater than a certain threshold, $|C_i| \geq \rho$). We assign node attribute values for the super-node of C_i by averaging the node attribute values of the nodes in C_i for numerical attributes. For categorical attributes, we perform label encoding and assign the most frequent values in C_i.* □

Fig. 2 shows the overall structure of the proposed CaaN 2L. The core idea is to construct the CaaN graph from the input graph and to utilize it at both local and global levels. In the figure, the input graph consists of three major communities, and blue nodes belong to minor communities. In global

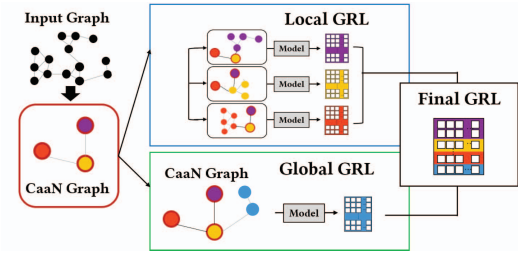


Fig. 2: The proposed CaaN 2L.

GRL, the CaaN graph is used to preserve global structural information. In local GRL, super-nodes in the CaaN graph are added to each subgraph, allowing each community to be learned while preserving connections with other communities. We integrate the results of the two levels to obtain a final embedding of the input graph.

Example 1. Let us assume that the input graph $G = \{n_1, ..., n_{100}\}$ contains three major communities, $C_1 = \{n_1, ..., n_{30}\}, C_2 = \{n_{31}, ..., n_{60}\}, C_3 = \{n_{61}, ..., n_{90}\}$, and other nodes $\{n_{91}, ..., n_{100}\}$ in minor communities. The CaaN graph G_{CaaN} consists of three super-nodes c_1, c_2, c_3, each corresponding to the major communities C_1, C_2, and C_3, respectively. In the global GRL phase, we learn G_{CaaN} with nodes in minor communities, $\{n_{91}, ...n_{100}\}$, and obtain a node embedding vector of shape $(13 * d)$. We then extract only the vector values corresponding to nodes $\{n_{91}, ...n_{100}\}$ to construct the final embedding. In the local GRL phase, we learn the nodes within each major community in parallel. To capture the relationship with the other communities, we augment the super-nodes in the CaaN graph to the nodes in each community. For example, for a community C_1, we consider not only the nodes $\{n_1, ..., n_{30}\}$ in the community but also super-nodes c_2 and c_3 to consider the relationship from nodes in C_1. We learn the nodes and super-nodes and obtain a node embedding vector of shape $(32 * d)$. We then extract only the vector values corresponding to nodes $\{n_1, ...n_{30}\}$ in the community to construct the final embedding. □

B. Global GRL

In global GRL, we aim to capture the global structural patterns of the entire graph by representing global relationships at the community level using the CaaN graph while significantly reducing learning overheard. Furthermore, we can capture the relationship between the super-nodes in the CaaN graph and nodes in minor communities, which cannot be effectively learned when local GRL is applied due to its limited size, improving the representation accuracy for the nodes in minor communities. We formulate the process of constructing global GRL as shown in Eq. (1), and any GRL model can be employed for $GRL()$.

$$GlobalGRL(G) = GRL(CaaN(G) \cup (\cup_{i=m+1}^{k} C_i)) \quad (1)$$

C. Local GRL

In local GRL, we aim to capture the elaborate differences in the relationships between nodes in the same community by focusing on that community. Furthermore, to maintain connections with other communities during this process, we enhance local GRL with the CaaN graph by connecting the nodes in each community c_i to the other super-nodes except for a super-node, v_{c_i}, corresponding to c_i. This results in a minimal overhead for the CaaN graph while further improving the representation accuracy. We formulate the process of constructing local GRL as shown in Eq. (2), and any GRL model can be employed for $GRL()$.

$$LocalGRL(G) = \cup_{i=1}^{m} GRL(CaaN(G) \setminus v_{c_i} \cup C_i) \quad (2)$$

D. Algorithm

Algorithm 1 shows the processing steps of CaaN 2L. First, it partitions the entire graph G into subgraphs $C = \{c_1, ..., c_k\}$ (line 1). Then, it constructs G_{CaaN} by converting each major community into a single super-node (line 2). We calculate the node attributes of the super-nodes by Definition 1 (lines 3-5). In global GRL, we learn the node embedding on G_{CaaN} with nodes in minor communities. The result vectors for nodes belonging to the minor community are stored in $\mathbf{v}_{V_{minor}}$ (lines 6-7). Then, in local GRL, each major community is expanded with the CaaN graph. We perform local GRL for major communities in parallel. The result vectors for nodes belonging to the major community C_m are stored in $\mathbf{v}_{V_{C_m}}$ (lines 8-11). Finally, we concatenate the result vectors of local and global GRL to obtain the final embedding (line 12).

Algorithm 1 CaaN 2L

Input: Graph G, partitioning strategy S
/* 1. CaaN Graph Construction */
1: According to S, partition G to $C = \{c_1, ..., c_k\}$
2: Generate G_{CaaN} by converting c_m in C_{major} into v_{c_m}
3: **if** node attributes are not empty **then**
4: Calculate the node attribute values for v_{c_m}
5: **end if**
/* 2. Global GRL with CaaN */
6: V_{minor} = all nodes in C_{minor}
7: $\mathbf{v}_{V_{minor}} = f(G_{CaaN} \cup V_{minor})$
/* 3. Local GRL with CaaN */
8: **for** each c_m in C_{major} **do** /* parallel processing */
9: V_{c_m} = all nodes in c_m
10: $\mathbf{v}_{V_{C_m}} = f((G_{CaaN} \setminus v_{c_m}) \cup V_{c_m})$
11: **end for**
/* 4. Final GRL */
12: $f(G) = [\mathbf{v}_{V_{minor}} | \cup_{i=1}^{m} \mathbf{v}_{V_{C_i}}]$
Output: An embedding matrix with shape $(|V| * d)$

IV. Performance Evaluation

A. Experimental Settings

1) Datasets: To evaluate our model, we use four real datasets of various scales, two social networks (Foursquare[1] and Flickr [8]) and two citation networks (PubMed [9] and

[1]https://sites.google.com/site/yangdingqi/home/foursquare-dataset

TABLE I: Real network dataset statistics

Dataset	# Nodes	# Edges	# Labels	# Attributes
Foursquare	114,324	607,333	N/A	5
Flickr	7,575	239,738	9	12,047
PubMed	19,717	44,338	3	500
CiteSeer	3,312	4,732	6	3,703

CiteSeer [10]), as summarized in Table I. They are homogeneous, undirected, and unweighted graphs with node attributes. Because Foursquare has no node label (marked with "N/A"), we use it only for link prediction.

We assign all the datasets exclusively to two downstream tasks: 1) link prediction and 2) node classification. Specifically, we conduct experiments for link prediction on Foursquare and CiteSeer, and for node classification on Flickr, PubMed, and CiteSeer, while distributing seven base models across them. Furthermore, to observe the effects of different datasets with the same base model, we conduct experiments on node classification on various datasets while fixing the base model as GraphSAGE.

2) Base models: We adopt seven GRL models: 1) four widely used GRL models, Node2Vec [11], GCN [5], GAT [12], and GraphSAGE [13], 2) two models that utilize community information for graph embedding, CNRL [4] and AnECI [14], and 3) one self-supervised GRL model, GCA [15]. Node2Vec and CNRL are the random walk-based models, and the others are the GNN-based models.

3) Experimental environments: We implement all codes using Python (version 3.6.13) and develop our parallel execution using the Python multiprocessing library. We implement community detection algorithms and the CaaN graph using the iGraph Python library (version 0.9.9). We use Word2Vec provided by the Gensim (version 4.1.2) library for Node2vec and GraphSAGE, GCN, and GAT provided by the StellarGraph library (version 1.2.1) implemented on TensorFlow Keras (version 2.6.2). We reproduce CNRL[2], AnECI[3], and GCA[4] based on the codes provided by their authors. Our source code is publicly available at https://github.com/parkjungha/Two-level-GRL.

For all experiments, we use a machine equipped with one Intel Xeon 20-core processor and 32 GB RAM running on Ubuntu 18.04.6 LTS. We show the effects of parallel processing using multiple CPU cores.

4) Evaluation metrics: To measure the representation accuracy, we use the AUC score for link prediction and the accuracy for node classification. To measure the learning efficiency, we use the elapsed time for learning a model. To consider the variations of each experiment, we execute each experiment three times and present its average and standard deviation.

5) Parameter settings: Table II shows the hyperparameter values used in our experiments. To show that our method is not dependent on a particular community detection algorithm, we employ five representative community detection algorithms: 1) label propagation [16], 2) multilevel modularity [17], 3) eigenvector-based [18], 4) random walk-based [19], and 5) infomap algorithms [20]. For fair comparisons, we set the same parameters for random walk-based and GNN-based models, respectively. For our method, we select 100 for the community size threshold, which yields the highest accuracy when varying from 1 to 1000. We also set the default number of CPU cores for parallel processing to 20. For the parameters tailored to CNRL, AnECI, and GCA, we use the default parameters provided by them.

[2]http://nlp.csai.tsinghua.edu.cn/%7Etcc/datasets/simplified_CNRL.zip
[3]https://github.com/Gmrylbx/AnECI
[4]https://github.com/CRIPAC-DIG/GCA

TABLE II: Experimental parameters.

	Parameters	Values
Common	Embedding dimension (d)	256
	Size threshold (ρ)	100
	Number of cores (k)	20
Random walk-based	(p, q)	(1.0,1.0)
	Walk length	80
	Window size	10
	Batch size	128
GNN-based	Epochs	50
	Drop out rate	0.3
	Optimizer	Adam
	Learning rate	0.001

B. Performance on Link Prediction

Link prediction aims to predict edges between two nodes. To cover various cases with the node embeddings learned through GRL, we use different prediction models for each GRL type: 1) a logistic regression model for random walk-based models and 2) a fully connected layer for GNN-based models. For learning, we randomly remove 10% edges from the graph to build the training set. Then, we use the remaining graph to train the GRL models. The testing set consists of the 10% removed edges as positive samples and the same number of unconnected node pairs as negative samples.

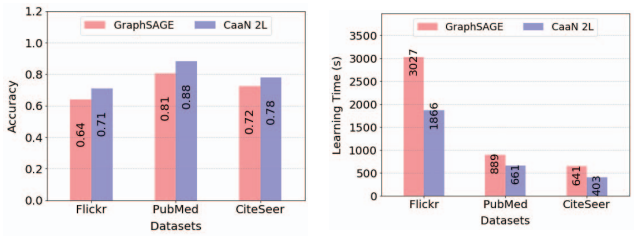
Table III summarizes link prediction results for seven base GRL models using the Foursquare and CiteSeer datasets. The results demonstrate that our model improves the representation accuracy of each base model while significantly reducing the learning time. Specifically, our model improves the accuracy by 1.35∼12.47% and reduces the learning time by 32.8∼48.1% across base models and datasets. It is worth noting that the accuracy improvement of our model based on AnECI is marginal compared to other base models. We claim that this is because AnECI's embeddings already preserve community information learned through GRL, but we note that our model further improves it.

C. Performance on Node Classification

Node classification aims to classify each node into one or multiple classes. The learned node embeddings are fed into the logistic regression model for random walk-based models and a fully connected layer for GNN-based models for multi-label classification. For learning, we randomly select 70% of nodes as the training set and the remaining nodes as the testing set.

TABLE III: Performance comparison on link prediction across base models and datasets

Dataset	Foursquare								CiteSeer					
Base model	GraphSAGE		Node2Vec		GCN		CNRL		GAT		GCA		AnECI	
Metric	Acc	Time (s)	Acc	Time (s)	Acc	Time (s)	Acc	Time (s)	Acc	Time (s)	Acc	Time (s)	Acc	Time (s)
Baseline	0.8478 ±0.002	6270 ±70	0.8491 ±0.007	5844 ±61	0.8353 ±0.2	7041 ±65	0.8033 ±0.003	6034 ±60	0.9080 ±0.002	667 ±35	0.9250 ±0.001	508 ±35	0.9345 ±0.001	555 ±30
CaaN 2L	0.9535 ±0.004	3254 ±83	0.9092 ±0.002	3750 ±65	0.8922 ±0.2	4803 ±73	0.8440 ±0.002	4028 ±65	0.9545 ±0.002	350 ±41	0.9622 ±0.001	329 ±19	0.9471 ±0.001	343 ±28
Improvements (%)	+12.47	+48.10	+5.83	+35.83	+6.81	+31.79	+5.07	+33.24	+5.12	+47.53	+4.02	+35.43	+1.35	+38.20



(a) Accuracy. (b) Learning time.

Fig. 3: Performance comparison for node classification on different datasets.

Table IV summarizes node classification results for seven base GRL models using the Flickr, PubMed, and CiteSeer datasets. The overall tendencies are quite similar to the results of link prediction in both accuracy and learning time.

Fig. 3 compares the performance on different datasets when the base model is fixed as GraphSAGE. Consistent with previous results, our model improves the accuracy by 7.89∼10.42% and reduces the learning time by 25.71∼38.35% compared to the base model.

D. Ablation Studies

To measure the effects of the CaaN graph in each level of our model, we conduct ablation studies by removing CaaN nodes in each level. By removing CaaN in local GRL, we mean that we run local GRL on each community only using its original nodes without super-nodes. By removing CaaN in global GRL, we mean that we run global GRL on the original input graph. Table V summarizes the accuracy and learning time results of ablation studies.

It demonstrates that our final model, as well as models without CaaN in each level, significantly improves the accuracy of the base model. These results reveal the effectiveness of using CaaN, even with one-level adoption. Furthermore, our final model outperforms both one-level adopted models. As expected, our final model significantly reduces the learning times of CaaN 2L without CaaN in global GRL. Compared to CaaN 2L without CaaN in local GRL, despite the slightly increased learning time, which is incurred by utilizing more nodes (i.e., super-nodes in CaaN) in addition to the original nodes in each community for local GRL, our model shows meaningful and consistent accuracy improvement.

V. RELATED WORK

A. Graph Representation Learning

Random walk-based models generate node sequences starting from random initial nodes and learn these sequences for each node. DeepWalk [3] applied the skip-gram of Word2Vec to the node sequences based on random walks [21]. Node2Vec [11] improved DeepWalk by using a more flexible sampling strategy.

GNN-based models apply neural networks to graph data. Kipf et al. [5] proposed graph convolutional networks (GCNs), which consider both graph structures and node labels based on layer-wise propagation. Since then, GCNs-based variants have emerged, including GraphSAGE [13], which samples neighboring nodes to reduce the learning overhead of GCNs, and GAT [12], which applies a self-attention mechanism to enhance the interpretability and performance of the model.

Recently, there has been a growing interest in self-supervised learning for GNN, which aims to learn representations of graph data without labeled data. Zhu et al. [15] proposed a graph contrastive representation learning method with adaptive augmentation (GCA), which highlights important connections and corrupts unimportant node features based on node centrality. Fang et al. [22] proposed a structure-preserving GRL model by maximizing mutual information between topological structure and feature representation using self-supervised approaches.

It is worth noting that several existing studies [23], [24] have introduced a concept similar to our CaaN, but with different purposes, i.e., subgraph embedding. Furthermore, they targeted small-sized graphs, with an average of 17 to 284 nodes, such as molecules. Li et al. [25] proposed CC-GNN that transforms each community into a node in a contracted graph for efficient and effective learning. However, they focused on the improvement of efficiency while sacrificing the original accuracy.

B. Community Detection

Raghavan et al. [16] proposed a label propagation-based algorithm, Blondel et al. [17] proposed a multilevel modularity optimization algorithm, Newman et al. [18] developed an eigenvector matrix-based algorithm, and Rosvall et al. [20] proposed an infomap algorithm minimizing the expected length of random walker trajectories. As an application of community detection, Park et al. [26] used community detection to identify relevant groups associated with cyberattack events and predicted cyberattacks based on these communities.

Several research efforts have attempted to enhance node embedding by incorporating community information. Cavallari et al. [27] proposed a framework for joint node- and community-level embedding with community detection. Liu et al. [14] proposed AnECI, which preserves community information by using a graph convolutional encoder with a new modu-

TABLE IV: Performance comparison on node classification across base models and datasets

Dataset	Flickr						PubMed				CiteSeer			
Base model	GraphSAGE		Node2Vec		GCN		GAT		GCA		CNRL		AnECI	
Metric	Acc	Time (s)	Acc	Time (s)	Acc	Time (s)	Acc	Time (s)	Acc	Time (s)	Acc	Time (s)	Acc	Time (s)
Baseline	0.6412 ±0.002	3027 ±64	0.5645 ±0.005	2990 ±68	0.6125 ±0.002	3569 ±62	0.7828 ±0.004	904.11 ±84	0.7957 ±0.002	747.12 ±40	0.6465 ±0.003	730.27 ±25	0.7169 ±0.002	404.15 ±21
CaaN 2L	0.7088 ±0.002	1866 ±59	0.6329 ±0.006	1773 ±48	0.7201 ±0.001	2042 ±44	0.8194 ±0.005	672.13 ±88	0.8310 ±0.003	485.22 ±29	0.6892 ±0.001	555.46 ±18	0.7358 ±0.003	277.06 ±40
Improvements (%)	+10.54	+38.35	+12.12	+40.70	+15.68	+42.79	+4.68	+25.66	+4.44	+35.05	+6.60	+23.94	+2.64	+31.45

TABLE V: Ablation performance analysis

Task	Link prediction				Node classification			
Dataset	Foursquare		CiteSeer		PubMed		Flickr	
Base Model	GraphSAGE		Node2vec		GAT		GCN	
Metric	Acc	Time (s)	Acc	Time (s)	Acc	Time (s)	Acc	Time (s)
Baseline	0.8478 ±0.002	6270 ±70	0.9109 ±0.004	551 ±65	0.7828 ±0.004	904.11 ±84	0.6125 ±0.002	3569 ±62
w/o CaaN in Local GRL	0.9405 ±0.002	3124 ±89	0.969 ±0.006	342 ±71	0.8088 ±0.002	666.41 ±67	0.7025 ±0.002	2034 ±64
w/o CaaN in Global GRL	0.9489 ±0.002	6255 ±65	0.9788 ±0.004	540 ±76	0.8141 ±0.004	988.5 ±45	0.7146 ±0.001	3823 ±79
CaaN 2L **(proposed)**	**0.9535 ±0.004**	3254 ±83	0.9783 ±0.003	345 ±80	**0.8194 ±0.001**	672.13 ±58	0.7201 ±0.001	2042 ±44

larity function. Tu et al. [4] proposed community-enhanced NRL (CNRL) that jointly embeds local node characteristics and global community patterns. We note that, because our method can use any type of existing GRL model as the base model, our method can even apply the previous methods [4], [14] as the base GRL model for further improvement in terms of efficiency and accuracy. We reported our experimental results using AnECI and CNRL as base models in Sec. IV.

VI. Conclusion and Discussion

In this paper, we proposed a new GRL model, CaaN 2L, which improves both representation accuracy and learning efficiency. We demonstrated its effectiveness by applying it to two representative graph analysis tasks, link prediction and node classification, and evaluating its performance on four real-world graph datasets. In this study, the proposed model was designed to leverage parallel processing. As the scale of graphs continues to increase, we must extend GRL methods to a distributed environment. Therefore, as a future research direction, we plan to extend the proposed model to a distributed environment. To achieve this, we need to investigate more sophisticated partitioning schemes that balance between nodes, which is hard to be achieved by community detection, while also considering network communication costs between the nodes.

Acknowledgment

This work was supported by the National Research Foundation of Korea(NRF) grant funded by the Korea government(MSIT) (No. 2022R1F1A1067008), and by the Basic Science Research Program through the National Research Foundation of Korea(NRF) funded by the Ministry of Education (No. 2019R1A6A1A03032119).

References

[1] Z. Wu *et al.*, "A comprehensive survey on graph neural networks," *IEEE transactions on neural networks and learning systems*, vol. 32, no. 1, 2020.

[2] B. Li and D. Pi, "Network representation learning: a systematic literature review," *Neural Computing and Applications*, vol. 32, no. 21, 2020.

[3] B. Perozzi, R. Al-Rfou, and S. Skiena, "Deepwalk: Online learning of social representations," in *Proc. of the 20th ACM SIGKDD*, 2014.

[4] C. Tu *et al.*, "A unified framework for community detection and network representation learning," *IEEE Transactions on Knowledge and Data Engineering*, vol. 31, no. 6, 2018.

[5] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," *arXiv preprint:1609.02907*, 2016.

[6] H. Yin *et al.*, "Algorithm and system co-design for efficient subgraph-based graph representation learning," *arXiv preprint:2202.13538*, 2022.

[7] L. Wu *et al.*, "Deep learning techniques for community detection in social networks," *IEEE Access*, vol. 8, 2020.

[8] X. Huang, J. Li, and X. Hu, "Label informed attributed network embedding," in *Proc. of the 10th ACM WSDM*, 2017.

[9] P. Sen *et al.*, "Collective classification in network data," *AI magazine*, vol. 29, no. 3, 2008.

[10] C. L. Giles, K. D. Bollacker, and S. Lawrence, "Citeseer: An automatic citation indexing system," in *Proc. of the third ACM conference on Digital libraries*, 1998.

[11] A. Grover and J. Leskovec, "node2vec: Scalable feature learning for networks," in *Proc. of the 22nd ACM SIGKDD*, 2016.

[12] P. Veličković *et al.*, "Graph attention networks," *arXiv preprint arXiv:1710.10903*, 2017.

[13] W. Hamilton, Z. Ying, and J. Leskovec, "Inductive representation learning on large graphs," *Advances in neural information processing systems*, vol. 30, 2017.

[14] Y. Liu *et al.*, "Robust attributed network embedding preserving community information," in *2022 IEEE 38th ICDE*, 2022.

[15] Y. Zhu *et al.*, "Graph contrastive learning with adaptive augmentation," in *Proc. of the Web Conference 2021*.

[16] U. N. Raghavan, R. Albert, and S. Kumara, "Near linear time algorithm to detect community structures in large-scale networks," *Physical review E*, vol. 76, no. 3, 2007.

[17] V. Blondel *et al.*, "Fast unfolding of community hierarchies in large networks, 1–6 (2008)," *arXiv preprint arXiv:0803.0476*.

[18] M. E. Newman, "Finding community structure in networks using the eigenvectors of matrices," *Physical review E*, vol. 74, no. 3, 2006.

[19] P. Pons and M. Latapy, "Computing communities in large networks using random walks," in *Computer and Information Sciences-ISCIS 2005: 20th International Symposium*, 2005.

[20] M. Rosvall and C. T. Bergstrom, "Maps of information flow reveal community structure in complex networks," *arXiv preprint physics.soc-ph/0707.0609*, 2007.

[21] T. Mikolov *et al.*, "Efficient estimation of word representations in vector space," *arXiv preprint arXiv:1301.3781*, 2013.

[22] Fang *et al.*, "Structure-preserving graph representation learning," in *ICDM*. IEEE, 2022.

[23] Q. Sun *et al.*, "Sugar: Subgraph neural network with reinforcement pooling and self-supervised mutual information mechanism," in *Proc. of the Web Conference 2021*.

[24] D. Kim and A. Oh, "Efficient representation learning of subgraphs by subgraph-to-node translation," *arXiv preprint arXiv:2204.04510*, 2022.

[25] Z. Li *et al.*, "Cc-gnn: A community and contraction-based graph neural network," in *ICDM*. IEEE, 2022.

[26] J.-H. Park and H.-Y. Kwon, "Cyberattack detection model using community detection and text analysis on social media," *ICT Express*, 2021.

[27] S. Cavallari *et al.*, "Learning community embedding with community detection and node embedding on graphs," in *CIKM*, 2017.